DATABASE PROJECT
TOPIC: EPOKA CLUBS SYSTEM

MEMBERS:
Viola Zeneli (Leader)
Sidrit Rrustemi
Noel Zani
Nikol Dalipi
Suada Terolli
Alesia Dardha

WORK DIVISON:

Viola Zeneli: ER Diagram, Relational Schema, Indexes and 25 Managerial Queries
Sidrit Rrustemi: Front End
Nikol Dalipi: SQL code (creation and instertion) for Advisor, Supervises and an UPDATE example
Noel Zani: SQL code (creation and instertion) for Students, participatesIn, isMemberOf and normalization example
Suada Terolli: SQL code (creation and instertion) for DOSEmployee, Reports and an UPDATE example
Alesia Dardha: SQL code (creation and instertion) for Club, Event and a DELETE example

PROJECT OUTLINE AND PLANNING

The project to develop a database for the Epoka Clubs system stems from the growing need for universities to foster extracurricular activities through student clubs while ensuring efficient management of these activities. By addressing the growing need for data organization and accessibility the project aims to support both the university's administrative staff and the students participating in various clubs.
The first step of our project was collecting information about Epoka Clubs, so that we had a better understanding before we created our own database. We had a discussion with the Student Clubs and Activities Specialist and later we explored the already created website for Epoka Clubs on the internet.
After that we discussed as a group and shared our opinions which entities to create and how they are related. After we all agreed on our groups er diagram, then Viola made it in ERDPlus. Based on the ERD, she also made the Relational Schema on ERDPlus.
Then the work was divided in this way :
Nikol, Suada and Alesia took each 2 entities from our ERD. They created the relations, inserted values and provided update and delete examples.
Noel took 3 entities and created the relations and inserted values for them. She also provided an example for normalization (changing 1 of our tables just to give an example, since all our tables were already normalized).
After they finished, Viola checked all the codes and joined them all together. Meanwhile Sidrit continued his work on front end part, so that when the sql was all finished by the girls, he could only connect the sql with his front end.
After Viola joined all the sql codes, she continued to create indexes for each table column so that the speed was increased when we perform select operations, since her next task was to create 25 managerial queries.
After all sql was finished, then Sidrit connect it with his front end, and our epoka clubs database and website are ready now.

RESEARCH AND IDEA CONSOLIDATION

Normalization:

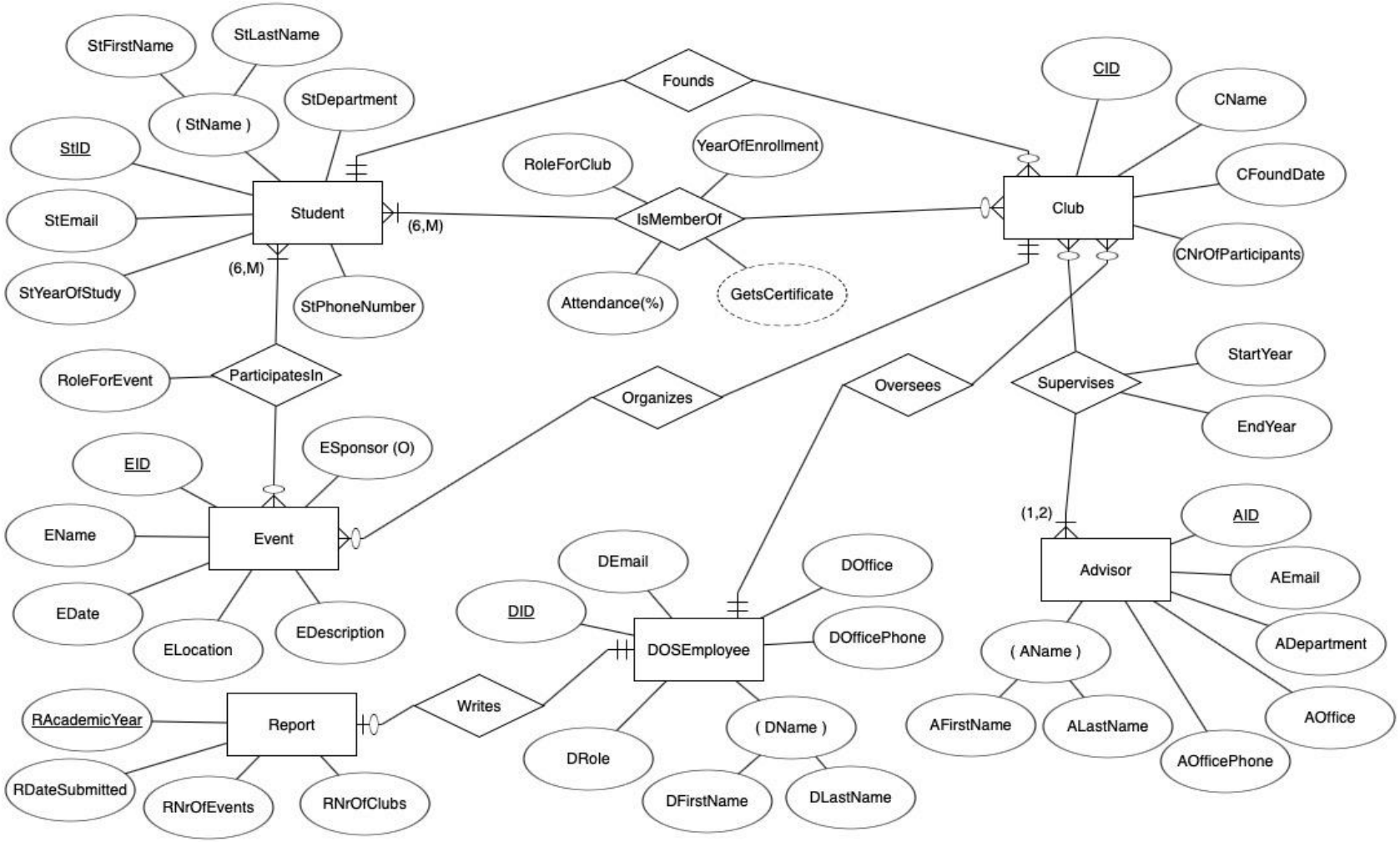
- Normalization avoids data duplication and inconsistencies, making the database faster, more reliable, and easier to maintain. Since all our tables were already normalized (on 3rd normal form), Noel provided an example how to normalize the table if it were not normalized.
- Our tables satisfy 3NF because the design eliminates redundancy, ensures all non-prime attributes depend directly on the primary key, and removes any dependencies between non-prime attributes.
- This ensures data integrity and prevents anomalies (e.g., insertion, update, and deletion anomalies), which are critical for maintaining a reliable and efficient database.

Indexing :

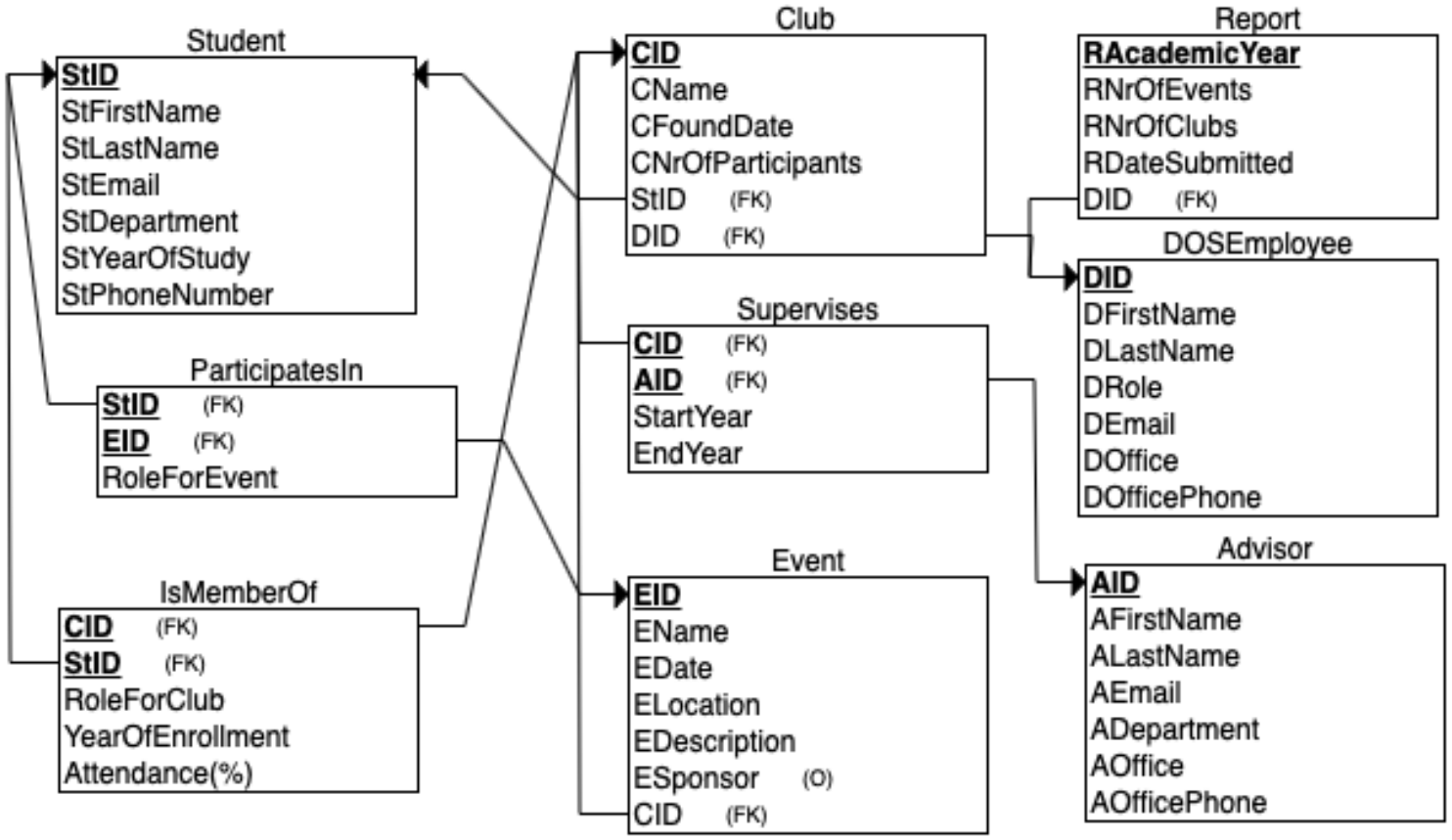
- As for the speed, indexing was implemented for each column ,by Viola, to enhance query performance, especially for frequently searched attributes such as Student IDs, Club IDs, and Event dates.
- Indexing is a critical feature in relational databases that improves the performance of query execution.

  Even though indexes consume extra storage, they allow the database to locate and access data more quickly instead of scanning the entire table for each query.

PROJECT EXECUTION
ERD DIAGRAM



RELATIONAL SCHEMA

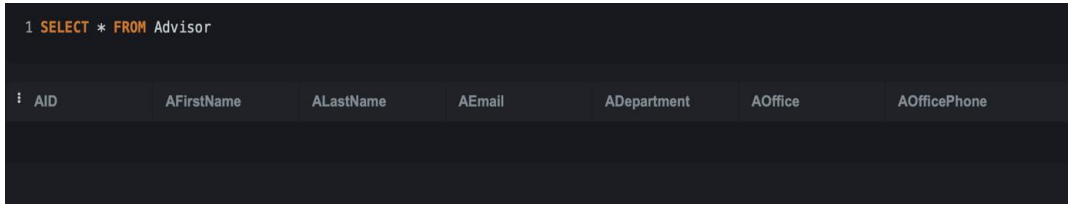


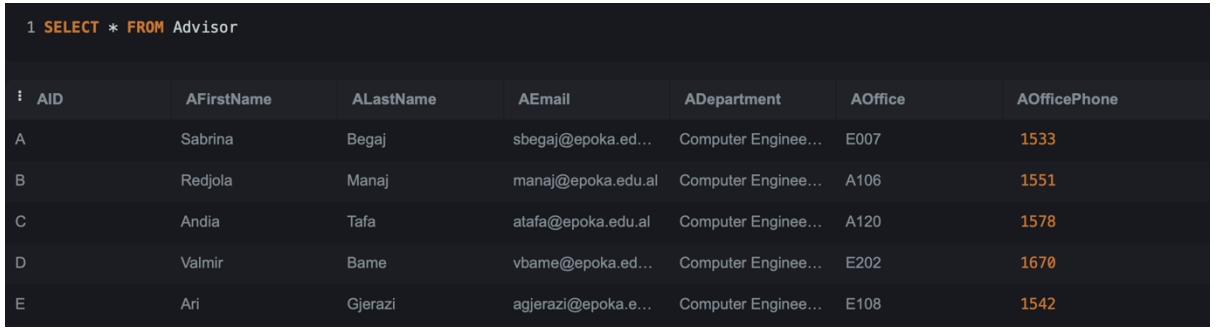
SQL CODE (Create relations, insert values, create indexes, make queries)

Advisor

```
CREATE TABLE Advisor(
AID CHAR PRIMARY KEY,
AFirstName VARCHAR(20) NOT NULL,
ALastName VARCHAR(20) NOT NULL,
AEmail VARCHAR(50) NOT NULL,
ADepartment VARCHAR(100) NOT NULL,
AOffice CHAR(4) NOT NULL,
AOfficePhone INT NOT NULL
);
```

```
INSERT INTO Advisor (AID, AFirstName, ALastName, AEmail,
ADepartment, AOffice, AOfficePhone) VALUES
('A', 'Sabrina', 'Begaj', 'sbegaj@epoka.edu.al', 'Computer
Engineering', 'E007', 1533),
('B', 'Redjola', 'Manaj', 'manaj@epoka.edu.al', 'Computer
Engineering', 'A106', 1551),
('C', 'Andia', 'Tafa', 'atafa@epoka.edu.al', 'Computer
Engineering', 'A120', 1578),
('D', 'Valmir', 'Bame', 'vbame@epoka.edu.al', 'Computer
Engineering', 'E202', 1670),
('E', 'Ari', 'Gjerazi', 'agjerazi@epoka.edu.al', 'Computer
Engineering', 'E108', 1542);
```





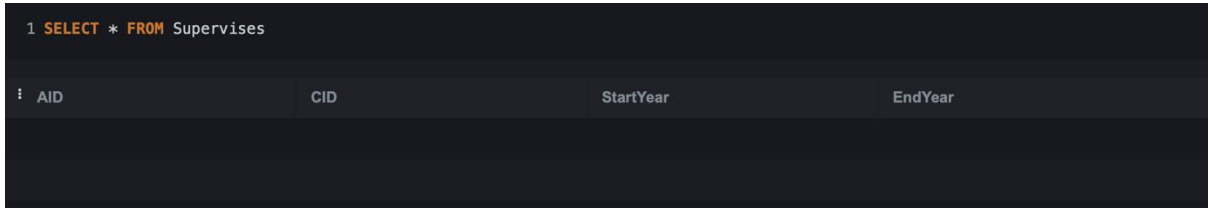
From this table, the dependency can be identified as:

AID -> AFirstName, ALastName, AEmail, ADepartment, AOffice, AOfficePhone (FKD)

Supervises

```
CREATE TABLE Supervises(
AID CHAR NOT NULL,
CID INT NOT NULL,
StartYear INT NOT NULL,
EndYear INT,
 PRIMARY KEY (AID, CID),
 FOREIGN KEY (AID) REFERENCES Advisor (AID) ON DELETE
CASCADE ON UPDATE RESTRICT,
 FOREIGN KEY (CID) REFERENCES Club (CID) ON DELETE
CASCADE ON UPDATE RESTRICT );
```

```
INSERT INTO Supervises (AID, CID, StartYear, EndYear) VALUES
('A', 5, 2021, NULL),
('C', 18, 2018, 2022),
('E', 9, 2023, NULL),
('B', 11, 2019, 2023),
('C', 7, 2024, NULL);
```





Advisor and Club hold a many-to-many relationship (though a club is usually supervised by 1 or 2 advisors).

From this table, the dependency can be identified as:
AID, CID -> StartYear, EndYear (FKD)

UPDATE EXAMPLE

Let’s suppose we want to update the Office Phone of one of the Advisors:

```
UPDATE Advisor
SET AOfficePhone = 1654
WHERE AOfficePhone = 1551;
```

The Office Phone of Advisor Redjola Manaj has been updated.

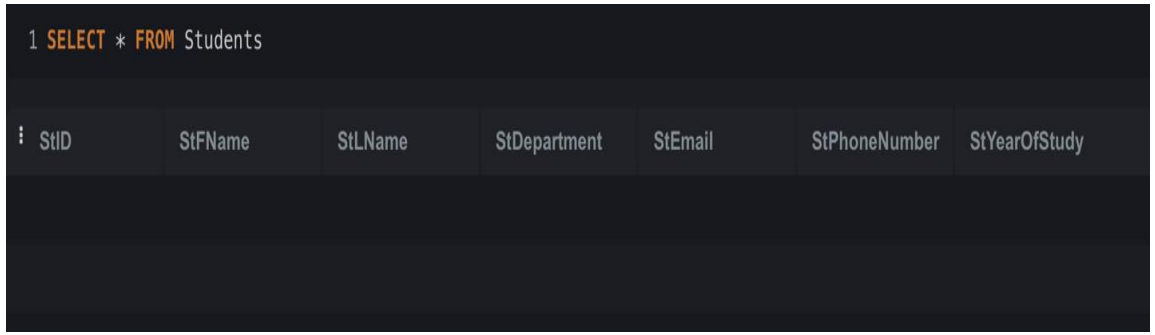
```
1 SELECT * FROM Advisor
```

AID	AFirstName	ALastName	AEmail	ADepartment	AOffice	AOfficePhone
A	Sabrina	Begaj	sbegaj@epoka.ed...	Computer Engineering	E007	1533
B	Redjola	Manaj	manaj@epoka.edu.al	Computer Engineering	A106	1654
C	Andia	Tafa	atafa@epoka.edu.al	Computer Engineering	A120	1578
D	Valmir	Bame	vbame@epoka.ed...	Computer Engineering	E202	1670
E	Ari	Gjerazi	agjerazi@epoka.e...	Computer Engineering	E108	1542

Students

```
CREATE TABLE Students
(StID CHAR(5) NOT NULL,
StFName VARCHAR (25) NOT NULL,
StLName VARCHAR (25) NOT NULL,
StDepartment VARCHAR (5) NOT NULL,
StEmail VARCHAR (20) NOT NULL,
StPhoneNumber VARCHAR(10) NOT NULL,
StYearOfStudy INT (1) NOT NULL,
PRIMARY KEY (StID) );
```

```
INSERT INTO Students(StID, StFName, StLName, StDepartment, StEmail, StPhoneNumber, StYearOfStudy) VALUES
('12345','Noel','Zani','SWE','nzani23@epoka.edu.al','0695436572','2'),
('11223','Viola','Zeneli','SWE','vzeneli23@epoka.edu.al','0693453496','2'),
('22453','Teddy','Daniels','BINF','tdaniels22@epoka.edu.al','0671253675','3'),
('25341','Nikol','Dalipi','SWE','ndalipi23@epoka.edu.al','0673916235','1'),
('12546','James','Conway','CEN','jconway24@epoka.edu.al','0691890744','1'),
('32546','Sidrit','Rrustemi','SWE','srrustemi23@epoka.edu.al','0695722508','2'),
('26743','Suada','Terolli','SWE','sterolli23@epoka.edu.al','0675647890','1'),
('19364','Joe','Pesci','IML','jpesci22@epoka.edu.al','0682501370','3'),
('37628','Alesia','Dardha','SWE','adardha23@epoka.edu.al','0691096341','2'),
('46378','Henry','Hill','CEN','hhill22@epoka.edu.al','0690749216','3');
```



1 SELECT * FROM Students

StID	StFName	StLName	StDepartment	StEmail	StPhoneNumber	StYearOfStudy
12345	Noel	Zani	SWE	nzani23@epoka....	695436572	2
11223	Viola	Zeneli	SWE	vzeneli23@epok...	693453496	2
22453	Teddy	Daniels	BINF	tdaniels22@epok...	671253675	3
25341	Nikol	Dalipi	SWE	ndalipi23@epoka...	673916235	1
12546	James	Conway	CEN	jconway24@epok...	691890744	1
32546	Sidrit	Rrustemi	SWE	srrustemi23@epo...	695722508	2
26743	Suada	Terolli	SWE	sterolli23@epoka...	675647890	1
19364	Joe	Pesci	IML	jpesci22@epoka....	682501370	3
37628	Alesia	Dardha	SWE	adardha23@epo...	691096341	2
46378	Henry	Hill	CEN	hhill22@epoka.e...	690749216	3

isMemberOf

```
CREATE TABLE isMemberOf
(StID CHAR (5) NOT NULL,
CID INT NOT NULL,
RoleForClub VARCHAR (15) NOT NULL,
YearOfEnrollment INT (4) NOT NULL,
Attendance   INT (3) NOT NULL,
PRIMARY KEY (StID, CID),
FOREIGN KEY (StId) REFERENCES Students (StId) ON DELETE
CASCADE ON UPDATE RESTRICT,
FOREIGN KEY (CID) REFERENCES Club (CID) ON DELETE
CASCADE ON UPDATE RESTRICT);
```

```
1 SELECT * FROM isMemberOf
```

StID	CID	RoleForClub	YearOfEnrollment	Attendance

```
INSERT INTO isMemberOf(StID, CID, RoleForClub, YearOfEnrollment, Attendance) VALUES
('25341','2','President','2024','100'),
('12546','2','VicePresident','2024','96'),
('37628','2','Member','2023','87'),
('26743','2','Member','2024','50'),
('11223','2','Secretary','2024','98'),
('12345','2','Treasurer','2023','90'),
('32546','2','Member','2024','95'),
('22453','2','Member','2022','68'),
('46378','5','Member','2022','78'),
('12345','5','President','2023','100'),
('19364','5','Member','2022','67'),
('11223','5','Treasurer','2023','70'),
('22453','5','VicePresident','2022','94'),
('26743','5','Member','2024','88'),
('32546','5','Secretary','2023','90');
```

```
1 SELECT * FROM isMemberOf
```

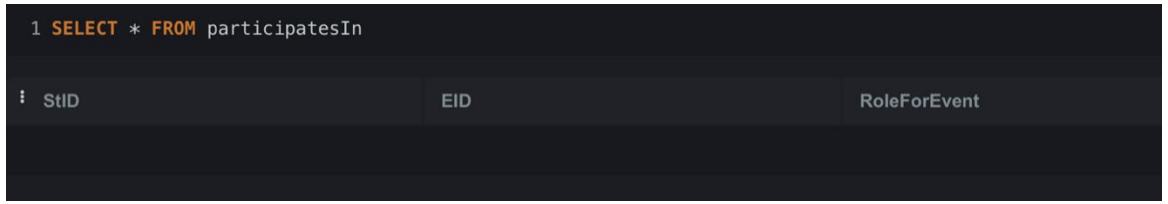
StID	CID	RoleForClub	YearOfEnrollment	Attendance
25341	2	President	2024	100
12546	2	VicePresident	2024	96
37628	2	Member	2023	87
26743	2	Member	2024	50
11223	2	Secretary	2024	98
12345	2	Treasurer	2023	90
32546	2	Member	2024	95
22453	2	Member	2022	68
46378	5	Member	2022	78
12345	5	President	2023	100
19364	5	Member	2022	67
11223	5	Treasurer	2023	70
22453	5	VicePresident	2022	94
26743	5	Member	2024	88
32546	5	Secretary	2023	90

1 student can be a member of more than 1 club.

1 club needs at least 6 students to be created.

participatesIn

```
CREATE TABLE participatesIn(
StID CHAR (5) NOT NULL,
EID CHAR NOT NULL,
RoleForEvent VARCHAR (15) NOT NULL,
PRIMARY KEY (StID,EID),
FOREIGN KEY (StID) REFERENCES Students (StID) ON DELETE CASCADE ON UPDATE RESTRICT,
FOREIGN KEY (EID) REFERENCES Event (EID) ON DELETE CASCADE ON UPDATE RESTRICT);
```




```
INSERT INTO participatesIn(StID, EID, RoleForEvent)VALUES
('32546','a','Organizator'),
('26743','a','Participant'),
('19364','a','Participant'),
('12345','a','Participant'),
('25341','a','Participant'),
('46378','a','Participant'),
('11223','a','Participant'),
('46378','c','Participant'),
('26743','c','Participant'),
('11223','c','Organizator'),
('22453','c','Participant'),
('37628','c','Participant'),
('19364','c','Participant');
```

1 SELECT * FROM participatesIn

StID	EID	RoleForEvent
32546	a	Organizator
26743	a	Participant
19364	a	Participant
12345	a	Participant
25341	a	Participant
46378	a	Participant
11223	a	Participant
46378	c	Participant
26743	c	Participant
11223	c	Organizator
22453	c	Participant
37628	c	Participant
19364	c	Participant

1 student can participate in more than 1 event. 1 event needs at least 6 students to happen.

NORMALIZATION

Let's suppose that there were dependencies in the Student table where StID is primary key and we had to normalize it. It would look like this:

StID	StFName	StLName	StDepartment	StEmail	StPhoneNumber	StYearOfStudy	StAdvisor	AdvisorOffice
12345	Noel	Zani	SWE	nzani23@epoka.edu.al	0695436572	2	Sabrina	E007
11223	Viola	Zeneli	SWE	vzeneli23@epoka.edu.al	0693453496	2	Sabrina	E007
22453	Teddy	Daniels	BINF	tdaniels22@epoka.edu.al	0671253675	3	Andia	A120
25341	Nikol	Dalipi	SWE	ndalipi23@epoka.edu.al	0673916235	1	Redjola	E007
12546	James	Conway	CEN	jconway24@epoka.edu.al	0691890744	1	Valmir	E202

From the table, the following dependencies can be identified:
1.StID → StFName, StLName, StDepartment, StEmail, StPhoneNumber, StYearOfStudy, StAdvisor (FKD)
2.StAdvisor → AdvisorOffice(TKD)

The table is already on the 1st form (No multivalues). ✓

2nd form(No PKD). ✓

3rd form(No TKD) we would have these tables:

In this table StAdvisor would be a primary key.

Students Table

StID	StFName	StLName	StDepartment	StEmail	StPhoneNumber	StYearOfStudy	StAdvisor
12345	Noel	Zani	SWE	nzani23@epoka.edu.al	0695436572	2	Sabrina
11223	Viola	Zeneli	SWE	vzeneli23@epoka.edu.al	0693453496	2	Sabrina
22453	Teddy	Daniels	BINF	tdaniels22@epoka.edu.al	0671253675	3	Andia
25341	Nikol	Dalipi	SWE	ndalipi23@epoka.edu.al	0673916235	1	Redjola
12546	James	Conway	CEN	jconway24@epoka.edu.al	0691890744	1	Valmir

Advisor Table

StAdvisor	AdvisorOffice
Sabrina	E007
Sabrina	E007
Andia	A120
Redjola	E007
Valmir	E202

CLUB

```
CREATE TABLE Club(
CID INT PRIMARY KEY,
CName VARCHAR(100) NOT NULL,
CFoundDate DATE NOT NULL,
CNrOfParticipants INT NOT NULL
);

INSERT INTO Club(CID, CName, CFoundDate, CNrOfParticipants)VALUES
(1,'Film Club','15-09-2018',480),
(2,'Young Talent Music Club','09-10-2020',340),
(3,'Sports Club','25-19-2015',675),
(4,'Google Developer Club','12-10-2019',1198),
(5,'Future Engineers Club','23-11-2018',956),
(6,'Volunteer Club','28-09-2016',1043),
(7,'Diplomacy Club','17-10-2023',180),
(8,'Young Economists Network','29-09-2022',215),
(9,'Finance Club','16-11-2017',940),
(10,'Programming Club','21-10-2016',1062),
(11,'Electronics Club','13-11-2018',867),
(12,'Innovators Club','29-11-2022',657),
(13,'Marketing Zone Club','12-10-2019',1035),
(14,'Sensitive Society Club','05-10-2023',726),
(15,'Iustitia Club','23-11-2018',740),
(16,'Ncep Club','16-10-2022',320),
(17,'Multimedia Club','21-10-2017',527),
(18,'Book Lovers Club','15-09-2021',614),
(19,'Theater Club','15-09-2021',462),
(20,'Podcast And Journal Club','24-10-2024',315),
(21,'Human Rights Defender Club','19-10-2024',214),
(22,'Art Club','20-10-2017',1010),
(23,'Womens Leadership Club','27-10-2021',826),
(24,'Robotic Club','02-11-2024',769),
(25,'Outdoors Club','31-10-2018',942);
```

From the table, the dependency can be identified as:

CID --> CName, CFoundDate, CNrOfParticipants (FKD)

```
1 SELECT * FROM Club
```

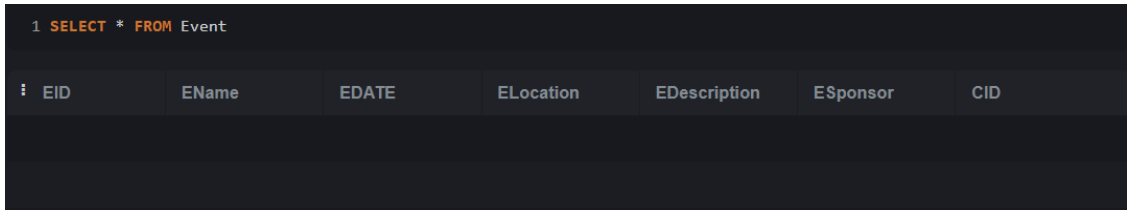
CID	CName	CFoundDate	CNrOfParticipants
1	Film Club	15-09-2018	480
2	Young Talent Music Club	09-10-2020	340
3	Sports Club	25-19-2015	675
4	Google Developer Club	12-10-2019	1198
5	Future Engineers Club	23-11-2018	956
6	Volunteer Club	28-09-2016	1043
7	Diplomacy Club	17-10-2023	180
8	Young Economists Network	29-09-2022	215
9	Fiance Club	16-11-2017	940
10	Programming Club	21-10-2016	1062
11	Electronics Club	13-11-2018	867
12	Innovators Club	29-11-2022	657
13	Marketing Zone Club	12-10-2019	1035
14	Sensitive Society Club	05-10-2023	726
15	Iustitia Club	23-11-2018	740
16	Ncep Club	16-10-2022	320

CID	CName	CFoundDate	CNrOfParticipants
17	Multimedia Club	21-10-2017	527
18	Book Lovers Club	15-09-2021	614
19	Theater Club	15-09-2021	462
20	Podcast And Journal Club	24-10-2024	315
21	Human Rights Defender Club	19-10-2024	214
22	Art Club	20-10-2017	1010
23	Womens Leadership Club	27-10-2021	826
24	Robotic Club	02-11-2024	769
25	Outdoors Club	31-10-2018	942

Event

```
CREATE TABLE Event(
EID CHAR PRIMARY KEY,
EName VARCHAR(100) NOT NULL,
EDATE DATE NOT NULL,
ELocation VARCHAR(100) NOT NULL,
EDescription VARCHAR(250) NOT NULL,
ESponsor VARCHAR(100),
CID INT,
FOREIGN KEY (CID) REFERENCES Club(CID) ON DELETE CASCADE ON UPDATE RESTRICT );
```

```
INSERT INTO Event(EID, EName, EDate, ELocation, EDescription, ESponsor, CID) VALUES
('a','Movie Screening: The Truman Show','19-01-2023','Conference Hall','Movie Screening next week',NULL,1),
('b','Art Week','23-03-2023','Epoka Campus','Showcasing local artists',NULL,22),
('c','Spring Fest','24-05-2023','Epoka Campus','Annual University Festival','Epoka University',2),
('d','Club Fest','04-11-2024','Epoka Campus','Get to know all the clubs offered at Epoka','x bank',2),
('e','DevFest','22-12-2024','Pyramid of Tirana','A meetup for tech enthusiasts','Google',4),
('f','Football League','15-10-2024','Soccer Field','All interested should apply for the teams',NULL,3),
('g','Poetry Competition','15-03-2023','A101','Call for all interested',NULL,22);
```




```
1 SELECT * FROM Event;
2
```

EID	EName	EDATE	ELocation	EDescripti…	ESponsor	CID
a	Movie Scre…	19-01-2023	Conference …	Movie Scre…	NULL	1
b	Art Week	23-03-2023	Epoka Cam…	Showcasing…	NULL	22
c	Spring Fest	24-05-2023	Epoka Cam…	Annual Univ…	Epoka Univ…	2
d	Club Fest	04-11-2024	Epoka Cam…	Get to know…	x bank	2
e	DevFest	22-12-2024	Pyramid of …	A meetup fo…	Google	4
f	Football Lea…	15-10-2024	Soccer Field	All intereste…	NULL	3
g	Poetry Com…	15-03-2023	A101	Call for all in…	NULL	22

From the table, the dependency can be identified as:

EID --> EName, EDate, ELocation, EDescription,ESponsor,CID (FKD)

In both table there's a Full Key Depndency so we don't have to do Normalization

Let’s suppose I want to delete the last row of the table Event. In that case I would write:

```
DELETE FROM Event
WHERE EID='g';
```

```
1 SELECT * FROM Event;
```

EID	EName	EDATE	ELocation	EDescripti…	ESponsor	CID
a	Movie Scre…	19-01-2023	Conference …	Movie Scre…	NULL	1
b	Art Week	23-03-2023	Epoka Cam…	Showcasing…	NULL	22
c	Spring Fest	24-05-2023	Epoka Cam…	Annual Univ…	Epoka Univ…	2
d	Club Fest	04-11-2024	Epoka Cam…	Get to know…	x bank	2
e	DevFest	22-12-2024	Pyramid of …	A meetup fo…	Google	4
f	Football Lea…	15-10-2024	Soccer Field	All intereste…	NULL	3

We see that the last row is deleted from the table

DOSEmployee

```
CREATE TABLE DOSEmployee (
DID VARCHAR(4) PRIMARY KEY,
DFirstName VARCHAR(25) NOT NULL,
DLastName VARCHAR(25)NOT NULL,
DRole VARCHAR(50) NOT NULL,
DEmail VARCHAR(77) NOT NULL,
DOffice CHAR(4)NOT NULL,
DOfficePhone INT
);
```

```
1 SELECT * FROM DOSEmployee;
2
```

DID	DFirstName	DLastName	DRole	DEmail	DOffice	DOfficePhone

From the table, the dependency can be identified as:

DID -> DFirstName, DLastName, Drole, DEmail, DOffice, DOfficePhone (FKD)

```
INSERT INTO DOSEmployee (DID, DFirstName, DLastName,
Drole, DEmail, DOffice, DOfficePhone) VALUES
('AA01','Sadullah', 'Yilmaz', 'Dean of Students',
'syilmaz@epoka.edu.al', 'A001', 111222333),
('AB02', 'Naida', 'Ertekin', 'Vice Dean of Students',
'nertekin@epoka.edu.al', 'E005', 123345567),
('AC03', 'Jonida', 'Sota', 'Coordinator', 'jsota@epoka.edu.al',
'E008', 432578975),
('AD04', 'Kelima', 'Hima', 'Career Planning and Alumni
Specialist', 'khima@epoka.edu.al', 'D104', 754231821),
('AE05', 'Jona', 'Kaso', 'Student Clubs and Activities Specialist',
'jkaso@epoka.edu.al', 'E002', 512721910);
```

```
1 SELECT * FROM DOSEmployee;
2
```

DID	DFirstName	DLastName	DRole	DEmail	DOffice	DOfficePhone
AA01	Sadullah	Yilmaz	Dean of Stu…	syilmaz@ep…	A001	111222333
AB02	Naida	Ertekin	Vice Dean o…	nertekin@e…	E005	123345567
AC03	Jonida	Sota	Coordinator	jsota@epok…	E008	432578975
AD04	Kelima	Hima	Career Plan…	khima@epo…	D104	754231821
AE05	Jona	Kaso	Student Clu…	jkaso@epok…	E002	512721910

Report

CREATE TABLE Report(
RAcademicYear CHAR(9) PRIMARY KEY,
RDateSubmitted DATE NOT NULL,
RNrOfEvents INT NOT NULL,
RNrOfClubs INT NOT NULL,
DID VARCHAR(4) NOT NULL,
FOREIGN KEY (DID) REFERENCES DOSEmployee ON DELETE CASCADE ON UPDATE RESTRICT);

```
1 SELECT * FROM Report;
```

RAcademicYear	RDateSubmitted	RNrOfEvents	RNrOfClubs	DID

From the table, the dependency can be identified as:

RAcademicYear -> RDateSubmitted, RNrOfEvents, RNrOfClubs, DID (FKD)

In both tables, there is Full Key Dependency, so we don't have to do normalization.

```
1 SELECT * FROM Report;
```

RAcademicYear	RDateSubmitted	RNrOfEvents	RNrOfClubs	DID
2017-2018	07/27/2018	34	10	AB02
2018-2019	08/26/2019	25	13	AA01
2019-2020	08/06/2020	33	20	AC03
2021-2022	07/21/2022	10	22	AE05
2022-2023	01/20/2023	45	24	AD04

INSERT INTO Report(RAcademicYear, RDateSubmitted, RNrOfEvents, RNrOfClubs, DID) VALUES
('2017-2018', '07/27/2018', 34, 10, 'AB02'),
('2018-2019', '08/26/2019', 25, 13, 'AA01'),
('2019-2020', '08/06/2020', 33, 20, 'AC03'),
('2021-2022', '07/21/2022', 26, 22, 'AE05'),
('2022-2023', '01/20/2023', 45, 24, 'AD04');

```
1 SELECT * FROM Report;
```

RAcademicYear	RDateSubmitted	RNrOfEvents	RNrOfClubs	DID
2017-2018	07/27/2018	34	10	AB02
2018-2019	08/26/2019	10	13	AA01
2019-2020	08/06/2020	33	20	AC03
2021-2022	07/21/2022	26	22	AE05
2022-2023	01/20/2023	45	24	AD04

UPDATE Report
SET RNrOfEvents=10
WHERE RNrOfEvents=26;

```
-- Indexes for Advisor
CREATE INDEX idx_advisor_aid ON Advisor(AID);
CREATE INDEX idx_advisor_afirstname ON Advisor(AFirstName);
CREATE INDEX idx_advisor_alastname ON Advisor(ALastName);
CREATE INDEX idx_advisor_aemail ON Advisor(AEmail);
CREATE INDEX idx_advisor_adepartment ON Advisor(ADepartment);
CREATE INDEX idx_advisor_aoffice ON Advisor(AOffice);
CREATE INDEX idx_advisor_aofficephone ON Advisor(AOfficePhone);

-- Indexes for Supervises
CREATE INDEX idx_supervises_aid ON Supervises(AID);
CREATE INDEX idx_supervises_cid ON Supervises(CID);
CREATE INDEX idx_supervises_startyear ON Supervises(StartYear);
CREATE INDEX idx_supervises_endyear ON Supervises(EndYear);

-- Indexes for Students
CREATE INDEX idx_students_stid ON Students(StID);
CREATE INDEX idx_students_stfname ON Students(StFName);
CREATE INDEX idx_students_stlname ON Students(StLName);
CREATE INDEX idx_students_stdepartment ON Students(StDepartment);
CREATE INDEX idx_students_stemail ON Students(StEmail);
CREATE INDEX idx_students_stphonenumber ON Students(StPhoneNumber);
CREATE INDEX idx_students_styearofstudy ON Students(StYearOfStudy);

CREATE INDEX idx_event_edescription ON Event(EDescription);
CREATE INDEX idx_event_esponsor ON Event(ESponsor);
CREATE INDEX idx_event_cid ON Event(CID);

-- Indexes for DOSEmployee
CREATE INDEX idx_dosemployee_did ON DOSEmployee(DID);
CREATE INDEX idx_dosemployee_dfirstname ON DOSEmployee(DFirstName);
CREATE INDEX idx_dosemployee_dlastname ON DOSEmployee(DLastName);
CREATE INDEX idx_dosemployee_drole ON DOSEmployee(DRole);
CREATE INDEX idx_dosemployee_demail ON DOSEmployee(DEmail);
CREATE INDEX idx_dosemployee_doffice ON DOSEmployee(DOffice);
CREATE INDEX idx_dosemployee_dofficephone ON DOSEmployee(DOfficePhone);

-- Indexes for Report
CREATE INDEX idx_report_racademicyear ON Report(RAcademicYear);
CREATE INDEX idx_report_rdatesubmitted ON Report(RDateSubmitted);
CREATE INDEX idx_report_rnrofevents ON Report(RNrOfEvents);
CREATE INDEX idx_report_rnrofclubs ON Report(RNrOfClubs);
CREATE INDEX idx_report_did ON Report(DID);
```

INDEX

To improve query performance, we will create indexes for all columns in each table.

Indexing is a critical feature in relational databases that improves the performance of query execution.

Even though indexes consume extra storage, they allow the database to locate and access data more quickly instead of scanning the entire table for each query.

```
-- Indexes for isMemberOf
CREATE INDEX idx_ismemberof_stid ON isMemberOf(StID);
CREATE INDEX idx_ismemberof_cid ON isMemberOf(CID);
CREATE INDEX idx_ismemberof_roleforclub ON isMemberOf(RoleForClub);
CREATE INDEX idx_ismemberof_yearofenrollment ON isMemberOf(YearOfEnrollment);
CREATE INDEX idx_ismemberof_attendance ON isMemberOf(Attendance);

-- Indexes for participatesIn
CREATE INDEX idx_participatesin_stid ON participatesIn(StID);
CREATE INDEX idx_participatesin_eid ON participatesIn(EID);
CREATE INDEX idx_participatesin_roleforevent ON participatesIn(RoleForEvent);

-- Indexes for Club
CREATE INDEX idx_club_cid ON Club(CID);
CREATE INDEX idx_club_cname ON Club(CName);
CREATE INDEX idx_club_cfounddate ON Club(CFoundDate);
CREATE INDEX idx_club_cnrofparticipants ON Club(CNrOfParticipants);

-- Indexes for Event
CREATE INDEX idx_event_eid ON Event(EID);
CREATE INDEX idx_event_ename ON Event(EName);
CREATE INDEX idx_event_edate ON Event(EDate);
CREATE INDEX idx_event_elocation ON Event(ELocation);
```

List of all final tables (to better understand how queries work) :

```
1 SELECT * FROM Students
```

StID	StFName	StLName	StDepartment	StEmail	StPhoneNumber	StYearOfStudy
12345	Noel	Zani	SWE	nzani23@epoka....	695436572	2
11223	Viola	Zeneli	SWE	vzeneli23@epok...	693453496	2
22453	Teddy	Daniels	BINF	tdaniels22@epok...	671253675	3
25341	Nikol	Dalipi	SWE	ndalipi23@epoka...	673916235	1
12546	James	Conway	CEN	jconway24@epok...	691890744	1
32546	Sidrit	Rrustemi	SWE	srrustemi23@epo...	695722508	2
26743	Suada	Terolli	SWE	sterolli23@epoka...	675647890	1
19364	Joe	Pesci	IML	jpesci22@epoka....	682501370	3
37628	Alesia	Dardha	SWE	adardha23@epo...	691096341	2
46378	Henry	Hill	CEN	hhill22@epoka.e...	690749216	3

```
1 SELECT * FROM Event;
```

EID	EName	EDATE	ELocation	EDescripti...	ESponsor	CID
a	Movie Scre...	19-01-2023	Conference ...	Movie Scre...	NULL	1
b	Art Week	23-03-2023	Epoka Cam...	Showcasing...	NULL	22
c	Spring Fest	24-05-2023	Epoka Cam...	Annual Univ...	Epoka Univ...	2
d	Club Fest	04-11-2024	Epoka Cam...	Get to know...	x bank	2
e	DevFest	22-12-2024	Pyramid of ...	A meetup fo...	Google	4
f	Football Lea...	15-10-2024	Soccer Field	All intereste...	NULL	3

```
1 SELECT * FROM isMemberOf
```

StID	CID	RoleForClub	YearOfEnrollment	Attendance
25341	2	President	2024	100
12546	2	VicePresident	2024	96
37628	2	Member	2023	87
26743	2	Member	2024	50
11223	2	Secretary	2024	98
12345	2	Treasurer	2023	90
32546	2	Member	2024	95
22453	2	Member	2022	68
46378	5	Member	2022	78
12345	5	President	2023	100
19364	5	Member	2022	67
11223	5	Treasurer	2023	70
22453	5	VicePresident	2022	94
26743	5	Member	2024	88
32546	5	Secretary	2023	90

```
1 SELECT * FROM participatesIn
```

StID	EID	RoleForEvent
32546	a	Organizator
26743	a	Participant
19364	a	Participant
12345	a	Participant
25341	a	Participant
46378	a	Participant
11223	a	Participant
46378	c	Participant
26743	c	Participant
11223	c	Organizator
22453	c	Participant
37628	c	Participant
19364	c	Participant

```
1 SELECT * FROM Supervises
```

AID	CID	StartYear	EndYear
A	5	2021	NULL
C	18	2018	2022
E	9	2023	NULL
B	11	2019	2023
C	7	2024	NULL

```
1 SELECT * FROM Advisor
```

AID	AFirstName	ALastName	AEmail	ADepartment	AOffice	AOfficePhone
A	Sabrina	Begaj	sbegaj@epoka.ed…	Computer Engineering	E007	1533
B	Redjola	Manaj	manaj@epoka.edu.al	Computer Engineering	A106	1654
C	Andia	Tafa	atafa@epoka.edu.al	Computer Engineering	A120	1578
D	Valmir	Bame	vbame@epoka.ed…	Computer Engineering	E202	1670
E	Ari	Gjerazi	agjerazi@epoka.e…	Computer Engineering	E108	1542

```
1 SELECT * FROM DOSEmployee;
```

DID	DFirstName	DLastName	DRole	DEmail	DOffice	DOfficePhone
AA01	Sadullah	Yilmaz	Dean of Stu…	syilmaz@ep…	A001	111222333
AB02	Naida	Ertekin	Vice Dean o…	nertekin@e…	E005	123345567
AC03	Jonida	Sota	Coordinator	jsota@epok…	E008	432578975
AD04	Kelima	Hima	Career Plan…	khima@epo…	D104	754231821
AE05	Jona	Kaso	Student Clu…	jkaso@epok…	E002	512721910

```
1 SELECT * FROM Report;
```

RAcademicYear	RDateSubmitted	RNrOfEvents	RNrOfClubs	DID
2017-2018	07/27/2018	34	10	AB02
2018-2019	08/26/2019	25	13	AA01
2019-2020	08/06/2020	33	20	AC03
2021-2022	07/21/2022	10	22	AE05
2022-2023	01/20/2023	45	24	AD04

```
1 SELECT * FROM Club
```

CID	CName	CFoundDate	CNrOfParticipants
1	Film Club	15-09-2018	480
2	Young Talent Music Club	09-10-2020	340
3	Sports Club	25-19-2015	675
4	Google Developer Club	12-10-2019	1198
5	Future Engineers Club	23-11-2018	956
6	Volunteer Club	28-09-2016	1043
7	Diplomacy Club	17-10-2023	180
8	Young Economists Network	29-09-2022	215
9	Fiance Club	16-11-2017	940
10	Programming Club	21-10-2016	1062
11	Electronics Club	13-11-2018	867
12	Innovators Club	29-11-2022	657
13	Marketing Zone Club	12-10-2019	1035
14	Sensitive Society Club	05-10-2023	726
15	Iustitia Club	23-11-2018	740
16	Ncep Club	16-10-2022	320
17	Multimedia Club	21-10-2017	527
18	Book Lovers Club	15-09-2021	614
19	Theater Club	15-09-2021	462
20	Podcast And Journal Club	24-10-2024	315
21	Human Rights Defender Club	19-10-2024	214
22	Art Club	20-10-2017	1010
23	Womens Leadership Club	27-10-2021	826
24	Robotic Club	02-11-2024	769
25	Outdoors Club	31-10-2018	942

25 MANAGERIAL QUERIES :

1.Students with the role president(in isMemberOf) for any of the clubs.

```
SELECT s.StFName AS FirstName, s.StLName AS LastName, c.CName AS ClubName
FROM Students s, isMemberOf im, Club c
WHERE s.StID = im.StID AND im.CID = c.CID AND im.RoleForClub = 'President';
```

FirstName	LastName	ClubName
Nikol	Dalipi	Young Talent Music Club
Noel	Zani	Future Engineers Club

2.Clubs with More Than 500 Participants Ordered by Name

```
SELECT CName AS ClubName, CNrOfParticipants AS Participants
FROM Club
WHERE CNrOfParticipants > 500
ORDER BY CName ASC;
```

ClubName	Participants
Google Developer Club	1198
Innovators Club	657
Iustitia Club	740
Marketing Zone Club	1035
Multimedia Club	527
Outdoors Club	942
Programming Club	1062
Robotic Club	769
Sensitive Society Club	726
Sports Club	675
Volunteer Club	1043
Womens Leadership Club	826

3.Events Organized by Clubs Found Before 2020 ordered from latest event date to newest event date.

```
SELECT e.EName AS EventName, c.CName AS ClubName, e.EDate AS EventDate
FROM Event e, Club c
WHERE e.CID = c.CID AND c.CFoundDate < '2020-01-01'
ORDER BY e.EDate DESC;
```

EventName	ClubName	EventDate
Spring Fest	Young Talent Music Club	24-05-2023
Art Week	Art Club	23-03-2023
DevFest	Google Developer Club	22-12-2024
Movie Screening: The Truman Show	Film Club	19-01-2023
Club Fest	Young Talent Music Club	04-11-2024

4.Average Attendance for Each Role in Clubs

```
SELECT RoleForClub AS Role, AVG(Attendance) AS AverageAttendance
FROM isMemberOf
GROUP BY RoleForClub;
```

Role	AverageAttendance
Member	76.14285714285714
President	100
Secretary	94
Treasurer	80
VicePresident	95

5.Advisors and the clubs they supervise

```
SELECT a.AFirstName AS FirstName, a.ALastName AS LastName, c.CName AS ClubName
FROM Advisor a,Supervises s,Club c
where a.AID = s.AID and s.CID = c.CID
ORDER BY a.AFirstName ASC;
```

FirstName	LastName	ClubName
Andia	Tafa	Diplomacy Club
Andia	Tafa	Book Lovers Club
Ari	Gjerazi	Fiance Club
Redjola	Manaj	Electronics Club
Sabrina	Begaj	Future Engineers Club

6.Events with more than 6 participants

```
SELECT e.EName AS EventName, COUNT(pi.StID) AS TotalParticipants
FROM Event e, participatesIn pi
where e.EID = pi.EID
GROUP BY e.EName
HAVING COUNT(pi.StID) > 6;
```

EventName	TotalParticipants
Movie Screening: The Truman Show	7

7.Top 3 Clubs with the Most Participants and their Supervisors

```
SELECT c.CName AS ClubName, c.CNrOfParticipants AS TotalParticipants,
    a.AFirstName AS SupervisorFirstName, a.ALastName AS SupervisorLastName
FROM Club c, Supervises s,Advisor a
where c.CID = s.CID and s.AID = a.AID
ORDER BY c.CNrOfParticipants DESC
LIMIT 3;
```

ClubName	TotalParticipants	SupervisorFirstName	SupervisorLastName
Future Engineers Club	956	Sabrina	Begaj
Fiance Club	940	Ari	Gjerazi
Electronics Club	867	Redjola	Manaj

8.Student with highest attendance for each club (that has participants)

```
SELECT im.CID AS ClubID, c.CName AS ClubName, im.StID AS StudentID,
    s.StFName AS StudentFirstName, s.StLName AS StudentLastName,
    im.Attendance AS HighestAttendance
FROM isMemberOf im
JOIN Students s ON im.StID = s.StID
JOIN Club c ON im.CID = c.CID
WHERE im.Attendance = (SELECT MAX(im2.Attendance) FROM isMemberOf im2
            WHERE im2.CID = im.CID)
ORDER BY c.CName;
```

ClubID	ClubName	StudentID	StudentFirst…	StudentLast…	HighestAttendance
5	Future Engine…	12345	Noel	Zani	100
2	Young Talent …	25341	Nikol	Dalipi	100

9.Clubs with events sponsored by Google

```
SELECT c.CName AS ClubName, e.EName AS EventName, e.EDate AS EventDate
FROM Club c ,Event e
WHERE c.CID = e.CID and e.ESponsor LIKE '%Google%'
ORDER BY e.EDate DESC;
```

ClubName	EventName	EventDate
Google Developer Club	DevFest	22-12-2024

10. Students that participate in clubs with above-average number of participants

```
SELECT s.StFName AS StudentFirstName, s.StLName AS StudentLastName, c.CName AS
ClubName
FROM Students s
JOIN isMemberOf im ON s.StID = im.StID
JOIN Club c ON im.CID = c.CID
WHERE c.CNrOfParticipants > (SELECT AVG(CNrOfParticipants) FROM Club)
ORDER BY s.StFName, s.StLName;
```

StudentFirstName	StudentLastName	ClubName
Henry	Hill	Future Engineers Club
Joe	Pesci	Future Engineers Club
Noel	Zani	Future Engineers Club
Sidrit	Rrustemi	Future Engineers Club
Suada	Terolli	Future Engineers Club
Teddy	Daniels	Future Engineers Club
Viola	Zeneli	Future Engineers Club

11.Students participating in events organized by the clubs they are member of

```
SELECT s.StID AS StudentID, s.StFName AS StudentFirstName, s.StLName AS StudentLastName,
e.EName AS EventName
FROM Students s
JOIN participatesIn pi ON s.StID = pi.StID
JOIN Event e ON pi.EID = e.EID
WHERE e.CID IN (
   SELECT im.CID
   FROM isMemberOf im
   WHERE im.StID = s.StID
)
ORDER BY s.StFName;
```

StudentID	StudentFirstName	StudentLastName	EventName
37628	Alesia	Dardha	Spring Fest
26743	Suada	Terolli	Spring Fest
22453	Teddy	Daniels	Spring Fest
11223	Viola	Zeneli	Spring Fest

12. List all student departments

```
SELECT DISTINCT StDepartment AS Department
FROM Students;
```

Department
BINF
CEN
IML
SWE

13.Students in clubs with more participants than average number of club participants

```
SELECT s.StID, s.StFName, s.StLName, c.CName AS ClubName
FROM Students s
JOIN isMemberOf im ON s.StID = im.StID
JOIN Club c ON im.CID = c.CID
WHERE c.CNrOfParticipants > (
   SELECT AVG(CNrOfParticipants)
   FROM Club
);
```

StID	StFName	StLName	ClubName
11223	Viola	Zeneli	Future Engineers Club
12345	Noel	Zani	Future Engineers Club
19364	Joe	Pesci	Future Engineers Club
22453	Teddy	Daniels	Future Engineers Club
26743	Suada	Terolli	Future Engineers Club
32546	Sidrit	Rrustemi	Future Engineers Club
46378	Henry	Hill	Future Engineers Club

14. Clubs with or without events

```
SELECT c.CID AS ClubID, c.CName AS ClubName, e.EName AS
EventName
FROM Club c
LEFT JOIN Event e ON c.CID = e.CID;
```

ClubID	ClubName	EventName
1	Film Club	Movie Screening: The Truman Sho
2	Young Talent Music Club	Spring Fest
2	Young Talent Music Club	Club Fest
3	Sports Club	Football League
4	Google Developer Club	DevFest
5	Future Engineers Club	NULL
6	Volunteer Club	NULL
7	Diplomacy Club	NULL
8	Young Economists Network	NULL
9	Fiance Club	NULL
10	Programming Club	NULL
11	Electronics Club	NULL
12	Innovators Club	NULL
13	Marketing Zone Club	NULL
14	Sensitive Society Club	NULL
15	Iustitia Club	NULL
16	Ncep Club	NULL
17	Multimedia Club	NULL
18	Book Lovers Club	NULL
19	Theater Club	NULL
20	Podcast And Journal Club	NULL
21	Human Rights Defender Club	NULL
22	Art Club	Art Week
23	Womens Leadership Club	NULL
24	Robotic Club	NULL
25	Outdoors Club	NULL

15.Students who are both event participants and club members

```
SELECT s.StID AS StudentID, s.StFName AS FirstName, s.StLName AS LastName
FROM Students s
JOIN participatesIn pi ON s.StID = pi.StID
INTERSECT
SELECT s.StID, s.StFName, s.StLName
FROM Students s
JOIN isMemberOf im ON s.StID = im.StID;
```

StudentID	FirstName	LastName
11223	Viola	Zeneli
12345	Noel	Zani
19364	Joe	Pesci
22453	Teddy	Daniels
25341	Nikol	Dalipi
26743	Suada	Terolli
32546	Sidrit	Rrustemi
37628	Alesia	Dardha
46378	Henry	Hill

16.Clubs that have members but do not host events (can be done also with minus keyword)

```
SELECT DISTINCT c.CID AS ClubID, c.CName AS ClubName
FROM Club c
JOIN isMemberOf im ON c.CID = im.CID
LEFT JOIN Event e ON c.CID = e.CID
WHERE e.EID IS NULL;
```

ClubID	ClubName
5	Future Engineers Club

17.Total members and average attendance for each club (that has members) sorted by average attendance in descending order

```
SELECT c.CID AS ClubID, c.CName AS ClubName, COUNT(im.StID) AS TotalMembers,
AVG(im.Attendance) AS AvgAttendance
FROM Club c
JOIN isMemberOf im ON c.CID = im.CID
GROUP BY c.CID
ORDER BY AvgAttendance DESC;
```

ClubID	ClubName	TotalMembers	AvgAttendance
2	Young Talent Music Club	8	85.5
5	Future Engineers Club	7	83.85714285714286

18.Total events organized by each club

```
SELECT c.CID as ClubID,c.CName AS ClubName, COUNT(e.EID) AS TotalEvents
FROM Club c
LEFT JOIN Event e ON c.CID = e.CID
GROUP BY c.CID
ORDER BY TotalEvents DESC;
```

ClubID	ClubName	TotalEvents
2	Young Talent Music Club	2
1	Film Club	1
3	Sports Club	1
4	Google Developer Club	1
22	Art Club	1
5	Future Engineers Club	0
6	Volunteer Club	0
7	Diplomacy Club	0
8	Young Economists Network	0
9	Fiance Club	0
10	Programming Club	0
11	Electronics Club	0
12	Innovators Club	0
13	Marketing Zone Club	0
14	Sensitive Society Club	0
15	Iustitia Club	0
16	Ncep Club	0
17	Multimedia Club	0
18	Book Lovers Club	0
19	Theater Club	0
20	Podcast And Journal Club	0
21	Human Rights Defender Club	0
23	Womens Leadership Club	0
24	Robotic Club	0
25	Outdoors Club	0

19.Students in Year 1 or Year 2, along with their club roles and respective club names

```
SELECT s.StID AS StudentID, s.StFName AS FirstName, s.StLName AS LastName,
im.RoleForClub AS Role, c.CName AS ClubName
FROM Students s
JOIN isMemberOf im ON s.StID = im.StID
JOIN Club c ON im.CID = c.CID
WHERE s.StYearOfStudy = 1
UNION
SELECT s.StID, s.StFName, s.StLName, im.RoleForClub, c.CName
FROM Students s
JOIN isMemberOf im ON s.StID = im.StID
JOIN Club c ON im.CID = c.CID
WHERE s.StYearOfStudy = 2
ORDER BY Role;
```

StudentID	FirstName	LastName	Role	ClubName
26743	Suada	Terolli	Member	Future Engineers Club
26743	Suada	Terolli	Member	Young Talent Music Club
32546	Sidrit	Rrustemi	Member	Young Talent Music Club
37628	Alesia	Dardha	Member	Young Talent Music Club
12345	Noel	Zani	President	Future Engineers Club
25341	Nikol	Dalipi	President	Young Talent Music Club
11223	Viola	Zeneli	Secretary	Young Talent Music Club
32546	Sidrit	Rrustemi	Secretary	Future Engineers Club
11223	Viola	Zeneli	Treasurer	Future Engineers Club
12345	Noel	Zani	Treasurer	Young Talent Music Club
12546	James	Conway	VicePresident	Young Talent Music Club

20.Show clubs and their number of participants for active(more than 300 participants) clubs (using a view)

```
CREATE VIEW ActiveClubs AS
SELECT CID, CName, CNrOfParticipants
FROM Club
WHERE CNrOfParticipants > 300;

SELECT * FROM ActiveClubs ;
```

CID	CName	CNrOfParticipants
20	Podcast And Journal Club	315
16	Ncep Club	320
2	Young Talent Music Club	340
19	Theater Club	462
1	Film Club	480
17	Multimedia Club	527
18	Book Lovers Club	614
12	Innovators Club	657
3	Sports Club	675
14	Sensitive Society Club	726
15	Iustitia Club	740
24	Robotic Club	769
23	Womens Leadership Club	826
11	Electronics Club	867
9	Fiance Club	940
25	Outdoors Club	942
5	Future Engineers Club	956
22	Art Club	1010
13	Marketing Zone Club	1035
6	Volunteer Club	1043
10	Programming Club	1062
4	Google Developer Club	1198

21.Retrieve Reports submitted by each DOSEmployee

```
SELECT
    CONCAT(D.DFirstName, ' ', D.DLastName) AS EmployeeName,
    R.RAcademicYear AS AcademicYear,
    R.RDateSubmitted AS DateSubmitted,
    R.RNrOfEvents AS NumberOfEvents,
    R.RNrOfClubs AS NumberOfClubs
FROM Report R, DOSEmployee D
where R.DID = D.DID
ORDER BY D.DFirstName;
```

AdvisorName	NumberOfClubs
Andia Tafa	2
Sabrina Begaj	1
Redjola Manaj	1
Ari Gjerazi	1

22.Retrieve advisors overseeing the most clubs

```
SELECT
    CONCAT(A.AFirstName, ' ', A.ALastName) AS AdvisorName,
    COUNT(S.CID) AS NumberOfClubs
FROM Advisor A, Supervises S
where A.AID = S.AID
GROUP BY A.AID
ORDER BY NumberOfClubs DESC;
```

EmployeeName	AcademicYear	DateSubmitted	NumberOfEvents	NumberOfClubs
Jona Kaso	2021-2022	07/21/2022	10	22
Jonida Sota	2019-2020	08/06/2020	33	20
Kelima Hima	2022-2023	01/20/2023	45	24
Naida Ertekin	2017-2018	07/27/2018	34	10
Sadullah Yilmaz	2018-2019	08/26/2019	25	13

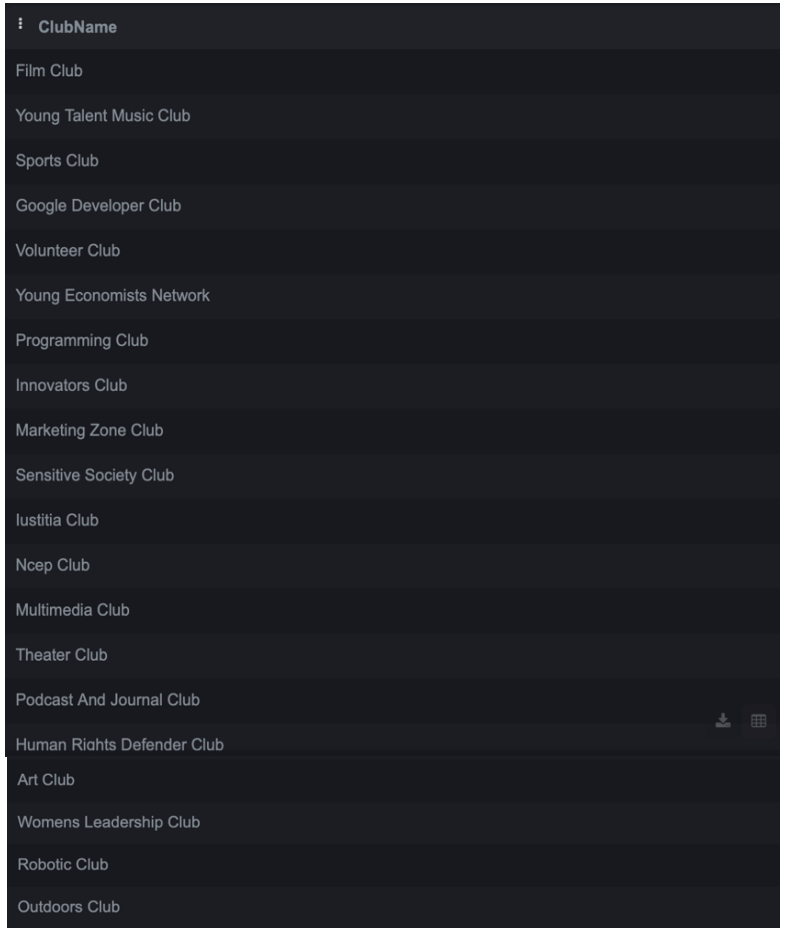
23. Retrieve events which had sponsors sorted from newest to oldest

```
SELECT E.EName AS EventName, E.EDate AS EventDate, E.ELocation AS Location, E.ESponsor AS Sponsor
FROM Event E
WHERE E.ESponsor IS NOT NULL
ORDER BY E.EDate DESC;
```

EventName	EventDate	Location	Sponsor
Spring Fest	24-05-2023	Epoka Campus	Epoka University
DevFest	22-12-2024	Pyramid of Tirana	Google
Club Fest	04-11-2024	Epoka Campus	x bank

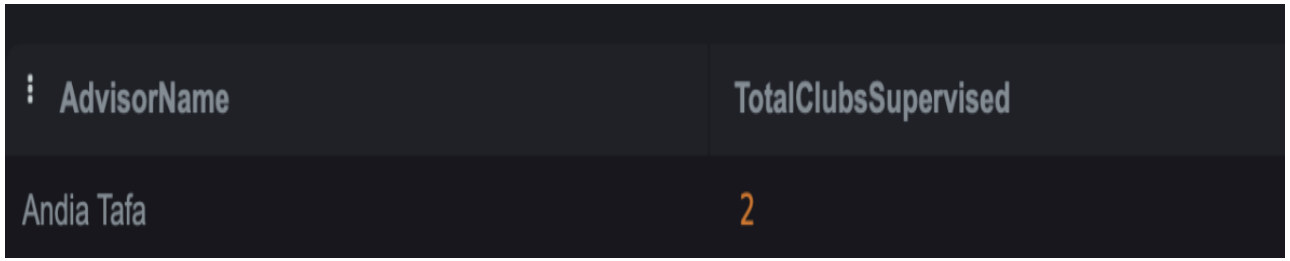
24. Retrieve all club names that have no advisor

```
SELECT C.CName AS ClubName
FROM Club C
LEFT JOIN Supervises S ON C.CID = S.CID
WHERE S.AID IS NULL
ORDER BY C.CID;
```

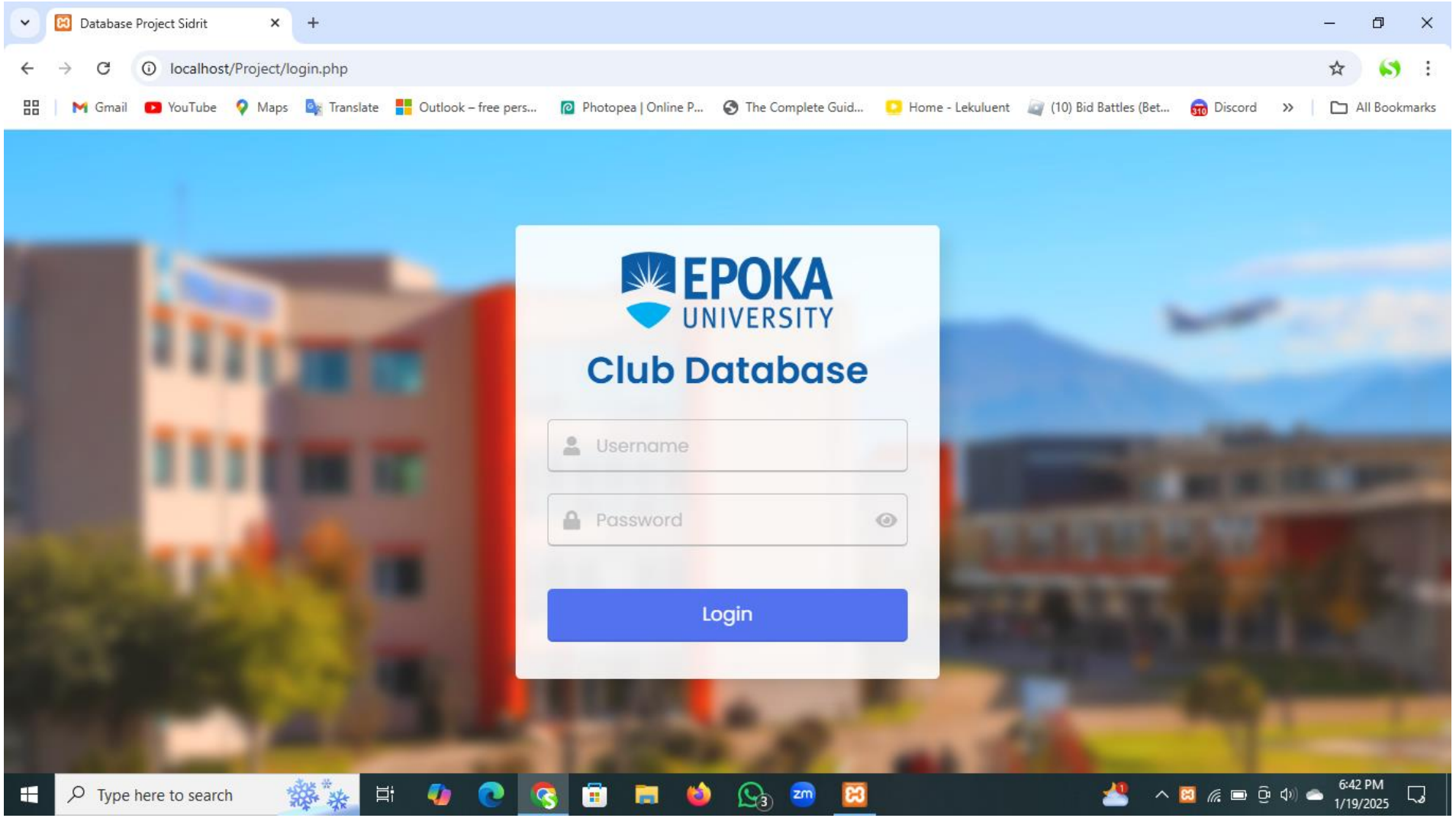


25. Retrieve the advisor supervising the highest number of clubs

```
SELECT CONCAT(A.AFirstName, ' ', A.ALastName) AS AdvisorName, COUNT(S.CID)
AS TotalClubsSupervised
FROM Advisor A, Supervises S
where A.AID = S.AID
GROUP BY A.AID
HAVING COUNT(S.CID) = (
      SELECT MAX(ClubCount) AS MaxClubCounts
      FROM (
         SELECT COUNT(CID) AS ClubCount
         FROM Supervises
         GROUP BY AID
      ) AS ClubCounts
);
```



FRONT END


Database Project Sidrit
localhost/Project/login.php
Gmail
YouTube
Maps
Translate
Outlook – free pers...
Photopea | Online P...
The Complete Guid...
Home - Lekuluent
(10) Bid Battles (Bet...
Discord
All Bookmarks
EPOKA
UNIVERSITY
Club Database
Username
Password
Login
Type here to search
6:42 PM
1/19/2025

THANK YOU !